

Videojuego de Sumo Pokémon

Informe proyecto final

1st Jesús Sebastian Mendoza
Departamento de Ingeniería
Universidad de Antioquia
Medellín, Antioquia
jsebastian.mendoza@udea.edu.co

2nd Danna Isabela Ochoa
Departamento de Ingeniería
Universidad de Antioquia
Medellín, Antioquia
danna.ochoa@udea.edu.co

I. INTRODUCCIÓN

El presente informe describe el diseño, desarrollo e implementación de un videojuego basado en el deporte del sumo, ambientado en un universo Pokémon. El proyecto fue desarrollado en C++ con Qt, integrando conceptos de programación orientada a objetos, herencia, polimorfismo, punteros, memoria dinámica, interfaz gráfica y modelos físicos. La propuesta enfrenta a dos combatientes con características distintas como fortaleza, agilidad o habilidades eléctricas, dentro de escenarios en los que deben empujarse, resistir impactos, evitar obstáculos y conservar sus vidas.

El videojuego consta de dos niveles principales. El Nivel 1 transcurre en una arena circular de sumo donde los personajes giran sobre su propio eje y deben impulsarse para sacar al rival del ring. El Nivel 2 presenta una vista lateral con una plataforma elevada en la que los jugadores pueden moverse, saltar, empujar al oponente, utilizar habilidades especiales y evitar caer al vacío.

II. PLANTEAMIENTO DEL PROBLEMA

El proyecto surge de la necesidad de construir un videojuego funcional que permita aplicar de manera concreta los temas abordados durante el curso: programación orientada a objetos, memoria dinámica, punteros, herencia, polimorfismo, interfaz gráfica con Qt y modelos físicos. El reto central consiste en transformar una idea deportiva en una experiencia interactiva donde personajes, obstáculos, niveles y eventos del escenario tengan comportamientos definidos mediante clases, atributos, métodos e interacciones entre objetos.

A partir de esto se plantea un juego en el que dos combatientes se enfrentan para sacar al rival del área de combate o hacerlo perder vidas mediante obstáculos y eventos del entorno. Con el fin de evitar que la dinámica resulte repetitiva, el juego se divide en dos niveles con mecánicas claramente diferenciadas: el primero, en una arena circular con rotación, impulso y expulsión del ring; el segundo, en una plataforma elevada de vista lateral con saltos, caídas, empujes, proyectiles y obstáculos. De esta manera, el software busca combinar jugabilidad, organización del código y aplicación de conceptos técnicos en una experiencia visualmente atractiva y coherente.

III. ESPECIFICACIÓN DE REQUERIMIENTOS

A. Requisitos funcionales

Inicio y navegación del juego: El videojuego debe permitir que el usuario inicie la aplicación desde una ventana principal, visualice el menú inicial, seleccione los personajes y acceda a la partida. La interfaz debe facilitar la transición del menú a las escenas de juego mediante `QGraphicsView`, mostrando las pantallas de transición correspondientes antes de iniciar cada nivel.

Selección de personajes: El sistema debe permitir elegir entre tres tipos de combatientes: fortachón, saltarín y eléctrico. Cada personaje debe poseer características propias que afecten su comportamiento durante la partida, como masa, fuerza, fricción, altura de salto, velocidad o habilidad especial.

Funcionamiento del Nivel 1: Este nivel se desarrolla en una arena circular de sumo. Los personajes giran sobre su propio eje y se impulsan para intentar sacar al rival del ring. Un jugador pierde vida cuando sale del área de combate o cuando es alcanzado por un obstáculo; en ese caso, el rival gana un punto.

Funcionamiento del Nivel 2: Este nivel se desarrolla en una plataforma elevada con vista lateral. Los jugadores deben poder moverse, saltar, empujarse, caer al vacío, utilizar habilidades especiales y ser afectados por obstáculos. Si un jugador cae de la plataforma, pierde una vida y el rival obtiene un punto.

Sistema de vidas y puntos: El juego debe mantener un registro de vidas y puntos para cada combatiente. Un jugador pierde vida al salir del ring, caer de la plataforma, ser alcanzado por un obstáculo dañino o ser aplastado por el rival. El oponente gana un punto cuando cualquiera de estas condiciones se cumple como resultado de una acción de combate o del entorno.

Obstáculos y eventos del escenario: El sistema debe incluir obstáculos que alteren la dinámica de la partida. Algunos, como el fuego o los rayos, causan pérdida de vida; otros, como el hielo, modifican la física del jugador al reducir la fricción y generar deslizamiento. Estos obstáculos deben integrarse a la lógica de cada nivel.

Habilidad del personaje eléctrico: El personaje eléctrico dispone de una habilidad especial basada en un rayo. En el

Nivel 2, este rayo actúa como proyectil que empuja al rival sin quitarle vida directamente; si el empuje provoca que el rival caiga, la pérdida de vida y la asignación del punto se procesan mediante la lógica de caída del nivel.

Agente inteligente: El juego debe incluir un agente inteligente representado por la clase `EntrenadorMaestroArena`. Este agente responde a condiciones del entorno y activa eventos del escenario mediante la gestión de obstáculos, como fuego, rayos y otros elementos que modifican la dinámica del combate.

B. Requisitos no funcionales

Usabilidad: El videojuego debe ser fácil de comprender y jugar. Los controles deben ser simples, la respuesta de los personajes debe ser inmediata y el usuario debe poder identificar con claridad los personajes, obstáculos, vidas, puntos y eventos relevantes durante la partida.

Rendimiento: El juego debe ejecutarse de forma fluida, con actualizaciones constantes de físicas, colisiones, movimiento de personajes, obstáculos, proyectiles y elementos visuales. La aplicación no debe bloquearse durante la partida ni presentar retrasos perceptibles en la respuesta a los controles.

Mantenibilidad: El código debe estar organizado en clases con responsabilidades bien definidas. La estructura debe permitir modificar niveles, personajes, obstáculos, sprites o sonidos sin afectar innecesariamente otras partes del sistema, lo que facilita la corrección de errores y futuras mejoras.

Escalabilidad: El sistema debe admitir la incorporación futura de nuevos personajes, niveles, obstáculos, sonidos o habilidades especiales. El uso de herencia, polimorfismo y clases base facilita extender el juego sin necesidad de reescribir la lógica existente.

Calidad visual y sonora: El videojuego debe mantener una estética coherente con la temática propuesta, utilizando fondos, sprites, plataformas, transiciones, música y efectos de sonido que refuercen la experiencia de juego. Los elementos visuales deben permitir distinguir con claridad a los personajes y los eventos del escenario.

C. Restricciones

Tecnología de desarrollo: El videojuego debe desarrollarse en C++ utilizando Qt, en concordancia con los requerimientos del curso y la necesidad de implementar una interfaz gráfica. El proyecto debe estar configurado mediante CMake y emplear los módulos Qt Widgets y Qt Multimedia.

Programación orientada a objetos: El proyecto debe aplicar herencia, polimorfismo, punteros y memoria dinámica. Cada clase debe representar un elemento relevante del juego, como personajes, niveles, obstáculos, físicas, agente inteligente o interfaz gráfica.

Modelos físicos: El videojuego debe incluir múltiples modelos físicos aplicados al funcionamiento del juego, entre ellos rotación, impulso lineal, velocidad angular, fricción, gravedad, detección de colisiones, caída desde plataforma y movimiento de proyectiles.

Recursos multimedia: Los sprites, fondos, plataformas, música y sonidos deben estar organizados dentro del proyecto y cargarse desde rutas válidas. Deben respetarse los derechos de autor de los recursos utilizados, especialmente para la presentación en video o tráiler.

D. Dependencias

Qt Widgets: El proyecto depende de este módulo para crear la ventana principal, gestionar botones, campos de texto, eventos de teclado, escenas gráficas, `QGraphicsView` y demás elementos visuales del juego.

Qt Multimedia: El videojuego depende de este módulo para reproducir música de fondo y sonidos de eventos, mejorando la experiencia del usuario mediante retroalimentación sonora durante la partida.

CMake: El proyecto utiliza CMake para configurar la compilación, organizar los archivos fuente y enlazar las bibliotecas necesarias de Qt, lo que permite mantener una estructura clara y facilitar la ejecución desde Qt Creator.

Recursos externos: El juego depende de archivos de imagen y audio necesarios para sostener la presentación visual y sonora del videojuego.

IV. METODOLOGÍA Y PLANIFICACIÓN

La metodología utilizada fue incremental e iterativa: el proyecto se construyó por etapas y se ajustó progresivamente según las necesidades de cada nivel. Primero se definió la idea general del juego, en nuestro caso sumo en un universo Pokémon, luego se establecieron las mecánicas principales, los personajes, las físicas comunes y la estructura general de clases. A partir de esa base, el equipo dividió el trabajo: se acordaron los aspectos compartidos, como físicas, lógica de personajes y organización del sistema, y cada integrante se encargó de desarrollar un nivel específico en profundidad.

Durante el proceso se aplicó un enfoque orientado a objetos en el que cada clase representa un componente importante del videojuego.

La planificación se desarrolló en cuatro fases. En la primera se definió la temática, los objetivos y la dinámica general de los dos niveles. En la segunda se diseñó la arquitectura del sistema, identificando las clases principales y sus relaciones. En la tercera se implementaron las físicas compartidas: rotación, impulso, fricción, gravedad, colisiones y movimiento de proyectiles. En la cuarta, cada nivel se desarrolló de forma independiente: el Nivel 1 se centró en la arena circular de sumo, y el Nivel 2 en la plataforma elevada con vista lateral, saltos, caída, obstáculos y habilidades especiales.

Finalmente, se realizó una fase de integración y pruebas en la que se incorporaron sprites, fondos, plataforma visual, música, sonidos, obstáculos y ajustes de jugabilidad. Durante esta etapa se verificó que los personajes se movieran correctamente, que las vidas y los puntos se actualizaran en los momentos adecuados, que los obstáculos afectaran a los jugadores y que las escenas se visualizaran correctamente en Qt. Esta forma de trabajo permitió avanzar por módulos, corregir errores de manera progresiva y mantener una separación clara

entre la lógica común del juego y los detalles particulares de cada nivel.

V. DIAGRAMA DE FLUJO

La Fig. 1 corresponde al diagrama de flujo, el cual contiene las clases implementadas en este proyecto.

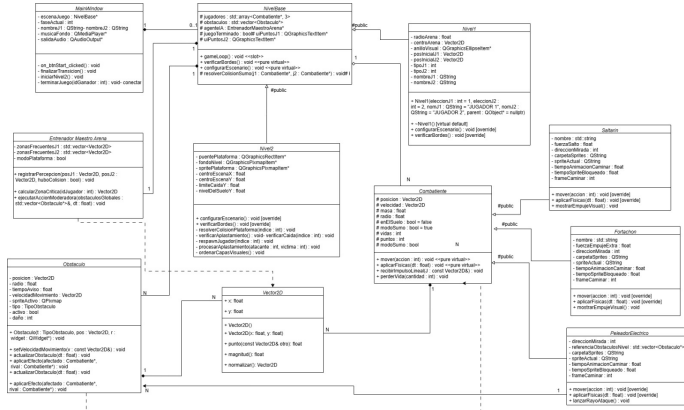


Fig. 1. Diagrama de flujo general del videojuego.

VI. DESARROLLO E IMPLEMENTACIÓN

El desarrollo del videojuego se realizó en C++ con Qt, integrando la lógica del juego con una interfaz gráfica basada en escenas y elementos visuales. Para ello se emplearon clases como `QGraphicsScene`, `QGraphicsView` y `QGraphicsItem`, que permitieron representar personajes, fondos, plataformas, obstáculos y proyectiles dentro de cada nivel. La ventana principal, gestionada desde `MainWindow`, se encarga de mostrar el menú inicial, permitir la selección de personajes y cargar la escena correspondiente.

La implementación se organizó mediante programación orientada a objetos, donde cada clase representa un componente específico del sistema. `Nivel1` y `Nivel2` contienen la lógica particular de cada escenario; `NivelBase` reúne el comportamiento común; `Combatiente` funciona como clase abstracta base para los personajes; `Fortachon`, `Saltarin` y `PeleadorElectrico` heredan de ella para definir características propias; `Obstaculo` modela los elementos que afectan a los jugadores; `Vector2D` gestiona posiciones, velocidades e impulsos; y `EntrenadorMaestroArena` cumple el rol de agente inteligente.

En el Nivel 1, la implementación se centró en una arena circular de sumo. Los jugadores giran sobre su propio eje mediante velocidad angular y pueden impulsarse para intentar sacar al rival del ring. Se aplicaron modelos físicos de rotación, impulso lineal, fricción, colisiones entre combatientes y detección de salida del área circular. Cuando un jugador sale del ring o es alcanzado por un obstáculo, pierde una vida y el rival gana un punto. El agente `EntrenadorMaestroArena` gestiona eventos como fuego, rayos y otros obstáculos del escenario, utilizando condiciones del entorno para intervenir en la partida.

En el Nivel 2 se implementó una dinámica diferente basada en una vista lateral con plataforma elevada. Los jugadores pueden desplazarse, saltar, empujarse, caer al vacío, aplastar al rival desde arriba y usar habilidades especiales. Se incorporaron físicas de gravedad, velocidad vertical, fricción, detección de colisión con la plataforma, caída y movimiento de proyectiles. El personaje eléctrico puede lanzar un rayo que empuja al oponente, mientras que los obstáculos de fuego, hielo o electricidad modifican la dinámica del combate.

La integración visual y sonora incluyó la adición de sprites para los personajes, fondos para los escenarios, plataforma visual, obstáculos gráficos, música de fondo y sonidos de eventos. Los sprites se cargan mediante `QPixmap`, mientras que la música y los efectos se gestionan con `Qt Multimedia`. Esto permitió que el videojuego, además de funcionar a nivel lógico, presentara una apariencia coherente y atractiva.

Durante la fase de pruebas se ajustaron la jugabilidad, las colisiones, la pérdida de vidas, la suma de puntos y la visualización correcta de elementos gráficos. También se realizaron ajustes en los tamaños de los personajes, las posiciones de la plataforma, los tiempos de aparición de obstáculos y el comportamiento de las habilidades especiales. Esta etapa permitió integrar las partes desarrolladas por separado y asegurar que ambos niveles compartieran una base común, pero conservaran mecánicas propias.

VII. DIFERENCIAS Y SIMILITUDES RESPECTO A MOMENTO I Y MOMENTO II

En comparación con el Momento I, la versión final conserva la idea principal del proyecto: un videojuego basado en luchas de sumo, donde dos personajes deben empujarse, colisionar y sacar al rival del área de combate. También se mantienen físicas importantes como rotación, impulso, fricción, colisiones, gravedad y saltos en el segundo nivel. El Nivel 1 sigue usando una arena circular con vista cenital, mientras que el Nivel 2 conserva la vista lateral con empujes, saltos y caídas.

La diferencia principal frente al Momento I está en la ambientación y en algunos elementos del diseño. Inicialmente el juego estaba pensado en un universo de Marvel, con escenarios como la Zona Negativa y el puente Bifrost, además de un agente tipo dron en el Nivel 2. En la versión final, la temática cambió a un universo inspirado en Pokémon, el agente inteligente pasó a representarse como 'EntrenadorMaestroArena' en el Nivel 1, y se agregaron personajes seleccionables como Fortachón, Saltarín y Peleador Eléctrico.

En comparación con el Momento II, la versión final se parece mucho más a lo planeado, porque ya se había definido el cambio hacia la temática Pokémon y la existencia de dos niveles diferenciados. Se mantuvo el Nivel 1 como combate en círculo de sumo con entrenadores y Pokémon, además del uso de obstáculos como fuego, hielo y rayos. También se conservó el Nivel 2 como una arena lateral con personajes diferenciados: Fortachón, Saltarín y Peleador Eléctrico.

Las diferencias frente al Momento II fueron más pequeñas y corresponden a ajustes de implementación. Por ejemplo, los power-ups no quedaron como una mecánica principal

completamente desarrollada, y el rayo del personaje eléctrico se implementó principalmente como un proyectil de empuje, no como daño directo. Además, algunas ideas se adaptaron para mejorar la jugabilidad y ajustarse al tiempo de desarrollo, pero la estructura general planteada en el Momento II sí se cumplió en la versión final.

VIII. RESULTADOS Y DISCUSIÓN

El funcionamiento logrado permite evidenciar una integración completa entre la lógica de programación, la interfaz gráfica y los modelos físicos implementados. Se desarrollaron dos niveles jugables con dinámicas diferenciadas: el Nivel 1 en una arena circular de sumo, y el Nivel 2 en una plataforma elevada con vista lateral. Además, se implementaron sistema de vidas y puntos, selección de personajes, obstáculos, sprites, fondos, música y sonidos, lo que resultó en una experiencia interactiva completa.

Uno de los logros más relevantes fue la aplicación de la programación orientada a objetos en la estructura del proyecto. La herencia permitió crear distintos tipos de personajes a partir de una clase base común, mientras que el polimorfismo facilitó tratarlos como combatientes genéricos aunque cada uno tuviera comportamientos particulares. El uso de punteros y memoria dinámica permitió crear objetos en tiempo de ejecución, como jugadores, obstáculos y elementos gráficos dentro de las escenas.

Entre las limitaciones actuales se identifican oportunidades de mejora en el balance entre personajes, la precisión de algunas colisiones y la organización de las rutas para recursos visuales y sonoros. Varios elementos dependen de rutas absolutas del equipo de desarrollo, lo que puede dificultar la ejecución en otros computadores si no se mantiene la misma estructura de carpetas. La retroalimentación visual del sistema de puntos, vidas y eventos también podría hacerse más explícita para el usuario.

Durante el desarrollo se enfrentaron dificultades principalmente en la integración entre las físicas y la capa gráfica. Fue necesario ajustar posiciones, tamaños, velocidades, fricción, gravedad, colisiones y límites de cada nivel para que el juego funcionara de manera coherente. La carga de sprites, fondos, música y sonidos también representó un reto, ya que estos recursos debían coincidir exactamente con las rutas del proyecto. Otro desafío fue lograr que ambos niveles compartieran una base común preservando mecánicas diferentes.

Como mejoras planeadas se propone optimizar el sistema de colisiones, mejorar el balance de los personajes, enriquecer los efectos visuales y sonoros, incluir una pantalla de ganador más elaborada y migrar las rutas de recursos a rutas relativas o archivos de recursos de Qt. El comportamiento del agente inteligente podría profundizarse para que tome decisiones más complejas según el comportamiento de los jugadores. En versiones futuras, el videojuego podría incorporar más niveles, nuevos personajes, nuevos obstáculos y una interfaz más completa con instrucciones y controles.

IX. CONCLUSIONES

El desarrollo de este videojuego permitió aplicar de manera práctica los conceptos principales del curso: programación orientada a objetos, herencia, polimorfismo, punteros, memoria dinámica, interfaz gráfica y modelos físicos. A través de la creación de personajes, niveles, obstáculos y eventos se pudo comprender cómo los elementos de un programa se organizan mediante clases con responsabilidades específicas, facilitando el mantenimiento y la extensión del sistema.

El proyecto también evidenció la importancia de la planificación y la división del trabajo. Al establecer una base común para las físicas y la estructura general, cada integrante pudo desarrollar un nivel con características propias sin perder coherencia con el resto del videojuego. Esta forma de trabajar facilitó integrar dos dinámicas distintas dentro de una misma aplicación: una arena circular con rotación e impulso, y una plataforma lateral con saltos, caídas y habilidades especiales.

El aprendizaje principal radica en que el desarrollo de un videojuego exige combinar lógica, diseño visual, física, interacción con el usuario y organización del código. No basta con que los personajes se desplacen; es indispensable que las reglas sean claras, que los eventos respondan correctamente y que la experiencia resulte comprensible para quien juega. Por ello, se recomienda continuar mejorando el proyecto mediante pruebas constantes, un mejor manejo de recursos, ajustes de jugabilidad y una mayor profundización en el agente inteligente y los efectos multimedia.

X. REFERENCIAS